

HDOC - EXPERIMENT 4

AIM

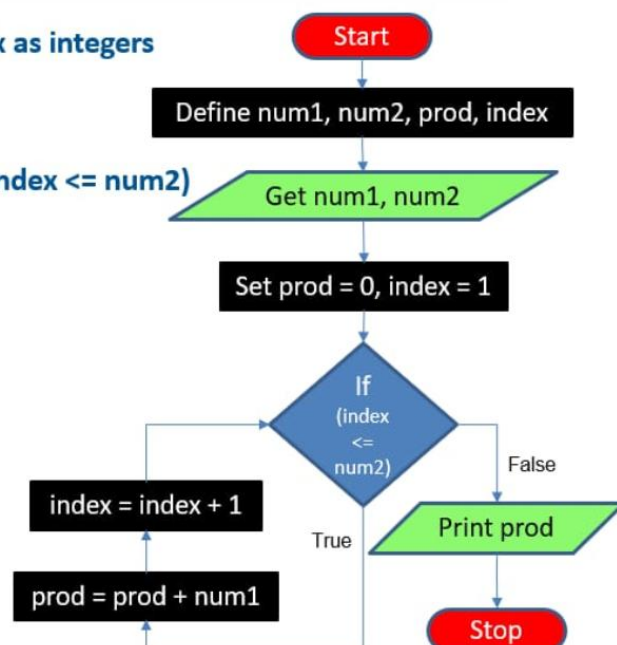
To identify mistakes in the given (incorrect) algorithm and flowchart for multiplying two integers without using the * operator, and to design, draw and verify a corrected algorithm and flowchart that uses the minimum possible number of additions and correctly handles all the sign combinations.

Given (incorrect) Algorithm and Flowchart

❖ Multiplying two integers without using * operator.

• Algorithm & Flowchart

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1. prod = prod + num1
 - 4.2. index = index + 1
5. Print prod
6. Stop



Faults Identified

❖ Algorithm

1. **No proper handling of signs:** The algorithm can never specify the signs of the product. It can never check whether num1 and num2 are negative or positive. Therefore, the result sign is never correct.

(e.g. $-6 \times 5 = -30$ but the product can never be negative as there is no such justification the logic.)

2. **Incorrect loop bound (not applicable on negative num2):** The loop provided “index $\leq$ num2” where index is equal to one and increasing is not applicable on negative num2. If num2 is negative the condition gets false on the very first check and the product remain zero giving incorrect output for the provided inputs.

(e.g. num2= -5 the condition “1 $\leq$ -5” gets false and the product remains 0.)

3. **No proper loop description:** “Repeat until” word indicates that the loop works till the condition becomes true and thus stops. But according to the flowchart it re-enters when the condition gets true and exists when it is false. Therefore, showcasing a while loop. Thus, the algorithm contradicts with the loops displaying two opposite loop semantics.

4. **Condition of minimum additions not followed:** The loop runs num2 times and adds num1 regardless which operand is smaller. This violates the condition of minimum possible additions.

(e.g. 12×3 adds 12 times instead only 3 are needed.)

5. **No proper handling for zero:** There is no condition to check whether (num1=0 or num2=0), the loop still runs num2 times adding zero num2 times providing a correct result but violation the condition of minimum possible additions.

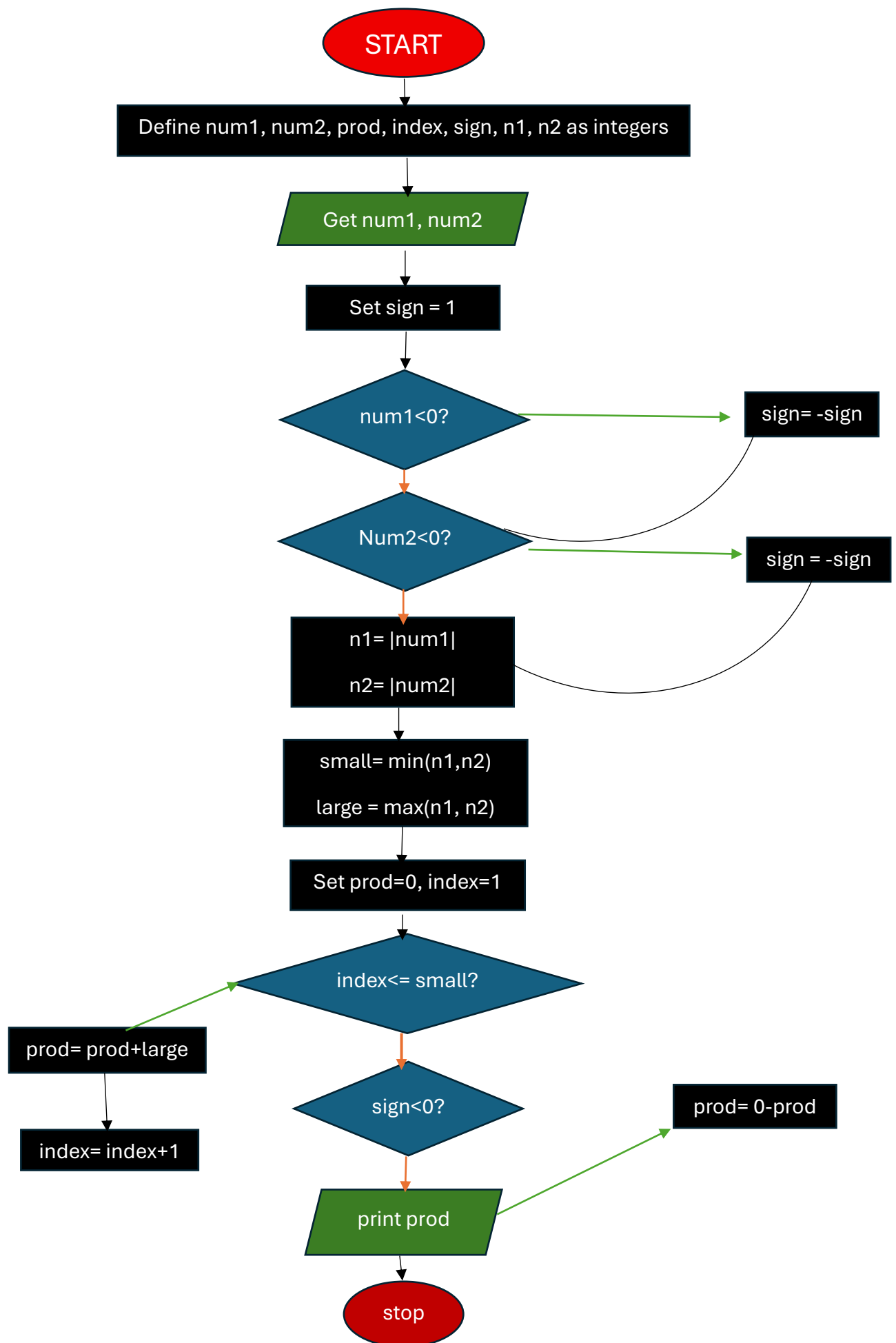
❖ Flowchart

1. **No sign handling block:** No decision box to determine the sign of num1 and num2 is provided therefore, just like the algorithm the output can never be negative given that one of the inputs is negative.
2. **Incorrect checking for decision box:** The diamond box directly checks the condition to num2 therefore index is always increasing to one so the product can never be true when num2 is 0 or negative.
3. **Loop doesn't adhere to minimum addition condition:** No step in the flowchart compares the num1 and num2 thus performing addition num2 times violating the condition of least minimum addition.
4. **Flowchart's true/false behaviour contradicts the algorithm:** The true condition enters loop again and false prints the product showcasing while loop whereas the algorithm works on "repeat until" loop.

Correct Algorithm

1. Define num1, num2, prod, index, sign, n1, n2 as integers.
2. Get num1 and num2.
3. Set sign = 1.
4. If (num1<0) set sign = -sign.
5. If (num2<0) set sign = -sign.
6. Set (n1 = |num1|) and (n2 = |num2|).
7. Set small = min(n1, n2) and large = max(n1, n2).
8. Set prod = 0, index = 1.
9. Repeat steps 9.1 and 9.2 while (index<=small).
 - 9.1 prod = prod +large
 - 9.2 index= index + 1
10. If (sign<0) then set prod = 0 – prod.
11. Print prod
12. Stop

Correct Flowchart



Verification against 13 cases

TC	num1	num2	n1, n2	small, large	sign	Obtained Output	Additions Used	Expected (HDOC table)	Result
1	12	3	12, 3	3, 12	+	Product: 36	3	Product: 36, 3 add.	PASS
2	3	12	3, 12	3, 12	+	Product: 36	3	Product: 36, 3 add.	PASS
3	-6	5	6, 5	5, 6	-	Product: -30	5	Product: -30, 5 add.	PASS
4	6	-5	6, 5	5, 6	-	Product: -30	5	Product: -30, 5 add.	PASS
5	-9	-4	9, 4	4, 9	+	Product: 36	4	Product: 36, 4 add.	PASS
6	-4	-9	4, 9	4, 9	+	Product: 36	4	Product: 36, 4 add.	PASS
7	0	8	0, 8	0, 8	+	Product: 0	0	Product: 0, 0 add.	PASS
8	8	0	8, 0	0, 8	+	Product: 0	0	Product: 0, 0 add.	PASS
9	0	-7	0, 7	0, 7	-	Product: 0	0	Product: 0, 0 add.	PASS
10	-7	0	7, 0	0, 7	-	Product: 0	0	Product: 0, 0 add.	PASS
11	0	0	0, 0	0, 0	+	Product: 0	0	Product: 0, 0 add.	PASS
12	1	-7	1, 7	1, 7	-	Product: -7	1	Product: -7, 1 add.	PASS
13	-1	-9	1, 9	1, 9	+	Product: 9	1	Product: 9, 1 add.	PASS